



Hyena Hierarchy

Towards Larger Convolutional Language Models

Bài báo: Poli, Massaroli, Nguyen, Dao, Baccus, Bengio, Ermon, Ré · Stanford & Mila · ICML 2023 (PMLR 202)

Nhóm 08 · CS2308

Trần Tú Quang · Tô Huỳnh Minh Tiến · Kiên

GVHD: TS. Nguyễn Văn Kiệt · University of Information Technology, VNU-HCM (UIT) · 2026



1. **Bối cảnh và động lực:** tại sao cần nghiên cứu
2. **Định nghĩa bài toán:** ba thuộc tính của attention và định nghĩa Hyena
3. **Đào sâu toán học:** recurrence, ma trận data-controlled, FFTConv, độ phức tạp
4. **Dữ liệu và đánh giá:** synthetic benchmark và benchmark chuẩn
5. **Kết quả và ảnh hưởng**
6. **Thực nghiệm:** Transformer-small và Hyena-small trên WikiText-2



Vì sao cần thay thế attention?

- Self-attention là trái tim của Transformer, nhưng có **chi phí bậc hai $O(L^2)$** theo độ dài chuỗi **L**.
- Chi phí này đặt **giới hạn cứng** lên lượng ngữ cảnh: khó dùng cả sách, nhạc dài, ảnh gigapixel, chuỗi DNA.
- Các giải pháp dưới bậc hai hiện có (Linformer, Reformer, Performer, sparse...) đều **phải lai ghép** với attention dày đặc mới đạt chất lượng, nên vẫn tồn tại **capability gap**.

Câu hỏi nghiên cứu trung tâm:

“Is attention all we need? Are there subquadratic operators that, inspired by its properties, are able to match its quality at scale?”

Mục tiêu: một toán tử **không dùng attention**, rẻ hơn, **không cần lai ghép**, vẫn sánh ngang Transformer ở quy mô lớn.



Vì sao các toán tử rẻ trước đây thua attention? Nhóm tác giả dùng **mechanistic interpretability** để truy ra 3 năng lực mấu chốt mà chúng thiếu:

Năng lực	SSM / Conv cố định	Attention
Phụ thuộc dữ liệu (data control)	Không, tĩnh	Có, $A(u)$
Ngữ cảnh không giới hạn	Không, thường bị locality	Có
Số tham số độc lập độ dài chuỗi	Có	Có

Hyena được thiết kế để **giữ đồng thời cả ba**: thay vì xấp xỉ attention, nhóm tác giả tái dựng các tính chất của nó bằng primitive rẻ hơn.



Ba thuộc tính cần bảo toàn

Phân rã self-attention: $y = A(q, k) v$, với $A(u) = \text{SoftMax}\left(\frac{1}{\sqrt{D}} u M_q M_k^\top u^\top\right)$.

1. **Data control:** $A(u)$ là toán tử tuyến tính **do dữ liệu quyết định**, một khối mã hóa cả họ hàm tuyến tính.
2. **Sublinear parameter scaling:** số tham số **tách rời** độ dài chuỗi, nhờ vậy dồn được tham số vào FFN.
3. **Unrestricted context:** không áp đặt locality, cho phép phụ thuộc tầm xa giữa bất kỳ hai vị trí.

Ý tưởng Hyena: thay vì xấp xỉ ma trận attention, hãy xây một toán tử **data-controlled** mới từ hai primitive rẻ: **long convolution** + **element-wise gating**.



Cho $N+1$ phép chiếu tuyến tính của đầu vào $(v, x^1, \dots, x^N)$ và N bộ lọc dài học được $h^1, \dots, h^N$:

$$\begin{aligned} z_t^1 &= v_t \\ z_t^{n+1} &= x_t^n (h^n * z^n)_t, \quad n = 1, \dots, N \\ y_t &= z_t^{N+1} \end{aligned}$$

- x_t^n là **gate** phụ thuộc đầu vào (nhân element-wise).
- $h^n * z^n$ là **long convolution** (bộ lọc dài bằng cả chuỗi).
- Khác attention: số phép chiếu **không bắt buộc bằng 3**; chiều sâu N của recurrence là siêu tham số điều khiển bậc của toán tử.

Short recurrences thu được các mô hình cũ (H3, GSS) như trường hợp đặc biệt.

$$(n * u) = \begin{bmatrix} \vdots & \vdots & \ddots & \vdots \\ h_{L-1} & h_{L-2} & \cdots & h_0 \end{bmatrix} \begin{bmatrix} \vdots \\ \vdots \\ \vdots \\ u_{L-1} \end{bmatrix} \quad (2)$$

2.1. Explicit and Implicit Convolutions

Parametrizing and optimizing convolution filters h_t is a standard procedure in deep learning and more broadly signal processing. The classical approach of *convolutional neural networks* (CNNs) (Fukushima and Miyake, 1982; LeCun et al., 1998; Ronneberger et al., 2015; He et al., 2016) is to optimize directly the values h_t of the filter's response at M prescribed steps, a parametrization we call *explicit*. M is referred to as the *filter size* and is typically much shorter than

Long convolutions and memory: A crude proxy for *memory* of a single computational unit is how far in the past it can access information to produce the output at a certain step. This can be roughly quantified by the number of non-zero entries $\partial y_t / \partial u_{t-n}$ for $n = 0, \dots, t$. The memory of CNNs filters is equivalent to the filter size M since $\partial y_t / \partial u_{t-n} = h_n$. The total mnemonic capacity of an all-convolutions CNN therefore scales with the number of model's parameters. Implicit parametrizations, on the other hand, allow us to disentangle the memory of each filter from the pa-



Từ recurrence sang dạng ma trận

Recurrence Hyena viết lại được dưới dạng **tích các ma trận data-controlled**:

$$y = H(u) v = D_x^N S_h^N \cdots D_x^2 S_h^2 D_x^1 S_h^1 v$$

- $D_x^n = \text{diag}(x^n) \in \mathbb{R}^{L \times L}$ là ma trận **đường chéo** (chính là gating).
- $S_h^n \in \mathbb{R}^{L \times L}$ là ma trận **Toeplitz** sinh bởi bộ lọc h^n (chính là convolution).

So sánh trực quan (Figure 2): SelfAttention $y = A(q, k)v$ là **một** ma trận dày data-controlled; còn Hyena là **tích xen kẽ** của đường chéo và Toeplitz, một phân rã thưa nhưng vẫn data-controlled, lấy cảm hứng từ **butterfly decomposition**.

u i.e., $A(u)$. We refer to operators of this type as *data-controlled*, as they encode a linear transformation $u \mapsto y$, that is, however, nonlinearly defined by u . This approach yields expressive nonlinear operators in u , and we hypothesize contributes, together with other mechanisms (Olsson et al., 2022), to the ability of certain operators to learn in-

ii. The matrix $H(u)$ is defined by interleaving implicit long convolutions and element-wise multiplication with one projection x^i at a time, until all projections are exhausted. Evaluation of $H(u)v$ is done efficiently **without materializing** $H(u)$. By doing so, we implicitly define a data-controlled operator as a factorization



H3 (Dao et al.) thực ra là một phân rã 3 thừa số:

$$A(q, k) = D_q S_\psi D_k S_\varphi, \quad H3(q, k, v) = A(q, k)v$$

- **Hyena bậc 2** tương đương cơ chế **H3**.
- **Hyena bậc 1** tương đương **GSS**, với một cách tham số hóa cụ thể cho bộ lọc (qua SSM).
- Hyena tổng quát hóa lên **bậc N tùy ý** và bộ lọc **dạng tự do (free-form)** thay vì buộc qua SSM.

Đây chính là ý nghĩa "hierarchy": một họ toán tử có thứ bậc, các mô hình trước là tầng thấp.



Tính trực tiếp $S_h v$ tốn $O(L^2)$. Mẹo: **chéo hóa ma trận tuần hoàn bằng cơ sở Fourier**.

$$\hat{S}_h = W^{-1} D_H W, \quad D_H = \text{diag}(\text{FFT}(h))$$

$$\text{pad}(y) = \text{iFFT}(\text{FFT}(\text{pad}(h)) \odot \text{FFT}(\text{pad}(u)))$$

- Zero-pad biến tích chập **tuyến tính** thành tích chập **vòng** (circular), tức nhân với ma trận tuần hoàn.
- Mỗi FFTConv tốn $O(L \log_2 L)$ và **không cần tạo ma trận $L \times L$ trong RAM**, nhờ vậy tránh tràn bộ nhớ (OOM).



Độ phức tạp toàn toán tử bậc N (Proposition 3.2):

$$\mathcal{O}(N D L (\log_2 L + D)) \ll \mathcal{O}(L^2)$$

Trực giác: đối ngẫu hai miền

- Convolution ở miền thời gian tương đương nhân element-wise ở miền tần số (và ngược lại).
- Hyena **luân phiên** giữa convolution (mở rộng "memory", gom ngữ cảnh rộng) và gating (nhân element-wise để lựa chọn thành phần tần số tinh tế).

Sự đan xen này là một giả thuyết lý giải vì sao Hyena hiệu quả: vừa nhìn xa, vừa lọc chọn lọc.



Bộ lọc dài **không** học như một vector tham số khổng lồ, mà **sinh ra từ một FFN nhỏ** (implicit parametrization):

$$h_t = \text{Window}(t) \cdot (\text{FFN} \circ \text{PositionalEncoding})(t)$$

- **Tách rời** độ dài bộ lọc khỏi số tham số (sublinear scaling).
- $\text{Window}(t) = \exp(-\alpha t)$ cho suy giảm mũ, điều hòa độ dài hiệu dụng; sine tần số cao trong FFN chống **low-frequency bias**.

Proposition 3.1 (Causal Hyenas): nếu mọi h^n nhân quả thì toán tử Hyena nhân quả, nhờ vậy huấn luyện tự hồi quy được (như mask tam giác của Transformer).



Xây dựng dữ liệu

(a) Synthetic, tự thiết kế để dò cơ chế (làm khó bằng độ dài và từ vựng lớn):

Tác vụ	Prompt	Target
Associative Recall	<code>a, 1, b, e, 3, f, b</code>	<code>e</code>
Majority	<code>a, g, g, g, e, f, g</code>	<code>g</code>
Counting	<code>a, b, b, b, a, c, b</code>	<code>4</code>
ICL of Functions	<code>$x_0, f(x_0), \dots, x_n$</code>	<code>$f(x_n)$</code>
Arithmetic	<code>1, 3, 5, +, 6, 8, 3</code>	<code>8, 1, 8</code>

Associative Recall đẩy tới **131k token** (lần đầu trình diễn ICL ở độ dài này).

(b) Benchmark chuẩn: WikiText103, The Pile (800GB), SuperGLUE, ImageNet-1k, CIFAR.



Chiều	Cách đo	Kết quả
Năng lực cơ chế	Acc.% trên synthetic	Duy nhất giải được recall; +>50đ vs SSM
Mô hình ngôn ngữ	Perplexity (WT103, Pile)	= Transformer, không hybrid
Hiệu quả	FLOPs , scaling law	đạt GPT với -20% FLOPs
Tốc độ	runtime (ms)	2× @8K, 100× @64K
Downstream	SuperGLUE, few-shot	sánh RWKV, ít token hơn
Thị giác	Acc. ImageNet/CIFAR	Hyena-ViT = ViT; Hyena-ISO 91.2%

Hyena bắt kịp attention tại $L \approx 2048$, bắt kịp FlashAttention tại $L \approx 4096$ đến 8192.



Tác động của nghiên cứu

- **Thách thức "attention is all you need"**: kiến trúc dưới bậc hai, đơn giản hơn, **có thể** sánh ngang Transformer ở quy mô dưới 1 tỷ tham số, dẫn tới nhận định **"attention may not be all we need."**
- **Mở đường ngữ cảnh siêu dài**: Hyena là nền cho **HyenaDNA**, **Evo** (bộ gene, chuỗi 10^5 đến 10^6 token) và **StripedHyena**, cộng hưởng với dòng **SSM**, **Mamba** ở những nơi attention bất khả thi.
- **Diễn giải cơ chế thành công cụ thiết kế**: benchmark recall và induction trở thành la bàn thiết kế kiến trúc, không chỉ để chấm điểm.
- **Primitive tổng quát**: dùng được ngoài ngôn ngữ, cho cả ảnh, âm thanh, sinh học.

Thông điệp kết: hiểu **vì sao** attention mạnh quan trọng hơn việc **sao chép** attention.



Phạm vi & mục tiêu

Nhóm không tái hiện toàn bộ paper, vì The Pile 800GB và GPU lớn vượt quá tài nguyên môn học. Thay vào đó nhóm làm tái hiện ở quy mô nhỏ, mục tiêu là kiểm chứng xu hướng chứ không khớp số tuyệt đối.

Hai câu hỏi nhóm muốn tự trả lời:

1. Ở quy mô nhỏ, perplexity của Hyena có ngang Transformer không?
2. Khi chuỗi dài ra, Hyena có lợi thế tốc độ như paper dự đoán không?

Code do nhóm tự cài bằng thuần PyTorch, có đối chiếu với bản chính chủ [HazyResearch/safari](#).



Dữ liệu: WikiText-2, tokenizer GPT-2, cắt chuỗi theo độ dài 256.

Cấu hình	Transformer-small	Hyena-small
Layers	4	4
<code>d_model</code> / <code>d_ff</code>	256 / 1024	256 / 1024
Trộn thông tin	4 heads attention	order N=2, FFTConv
Params	16.1M	16.3M

Hyena dùng `torch.fft.rfft` thuần PyTorch, không có custom CUDA kernel.

Huấn luyện bằng AdamW kèm warmup rồi cosine, đánh giá bằng val loss quy ra perplexity, chạy trên Colab T4.



Model	Val loss	Val PPL
Transformer-small	5.69	295.6
Hyena-small	5.53	251.8

Kết quả sơ bộ sau 5 epoch, chạy bằng notebook [notebooks/colab_E1_run.ipynb](#). Loss vẫn đang giảm nên đây là số minh họa xu hướng, chưa phải mức hội tụ cuối.

Nhận xét: hai model cùng cỡ tham số cho perplexity cùng tầm, Hyena còn nhỉnh hơn một chút. Đủ để nói Hyena là một baseline ngôn ngữ hợp lệ ở quy mô này.

Đo thời gian forward với input giả, cùng cấu hình model 16M tham số, batch 8, trên Colab T4. Tăng dần độ dài L:

L	Transformer (ms)	Hyena (ms)	Tỉ lệ
256	21.4	21.9	Hyena chậm hơn chút
512	43.7	42.4	hai bên hòa nhau
1024	100.1	84.1	Hyena nhanh hơn ~1.19 lần

Ở $L=2048$ Hyena vẫn chạy tốt (khoảng 168 ms) trong khi Transformer bắt đầu đuối. Attention tăng theo bình phương độ dài, còn Hyena tăng gần tuyến tính, nên chuỗi càng dài Hyena càng lợi.



- Ở chuỗi ngắn Hyena chậm hơn, vì chi phí FFT và đệm 2L lớn hơn phần tiết kiệm được. Hyena hòa ở khoảng L bằng 512 và vượt lên từ L bằng 1024, đúng như lưu ý trong paper.
- Bản cài của nhóm có đơn giản hóa so với safari: filter dùng SiLU thay cho activation dạng sin, modulation rút gọn, bỏ skip-connection. Vì vậy hội tụ có thể chậm hơn bản gốc.
- Vì dùng thuần PyTorch và không có CUDA kernel, nhóm chưa đạt mức speedup tuyệt đối như paper.
- Phần đo tốc độ dùng input giả để đo thời gian forward, không phải đánh giá perplexity trên tập validation.
- Cần phân biệt rõ: số trên WikiText-103, The Pile và speedup từ 8K đến 64K là của paper gốc, còn số WikiText-2 ở đây là của nhóm.



- Nhóm tự cài Hyena bằng thuần PyTorch, phần lõi khớp với bản chính chủ ở FFTConv đệm 2L, short conv depthwise và gating đệ quy bậc N.
- Quan sát đúng xu hướng chính của bài: tốc độ Hyena tăng gần tuyến tính, attention tăng theo bình phương, hai đường giao nhau quanh L bằng 512 và Hyena vượt lên rõ từ L bằng 1024.
- Bài học rút ra: hiểu được vì sao attention nghẽn ở chuỗi dài, và thấy được giới hạn thực tế của tái hiện khi thiếu kernel tối ưu và tài nguyên.

Quy mô nhỏ nhưng đủ để thấy tận mắt cơ chế dưới bậc hai mà bài báo đề xuất.



Cảm ơn! · Q&A

Nhóm 08 · CS2308

Trần Tú Quang · Tô Huỳnh Minh Tiến · Kiên

Tài liệu: Poli et al., [Hyena Hierarchy](#), ICML 2023 · docs/poli23a.pdf

Tài liệu nhóm: README.md · notebooks/colab_E1_run.ipynb